

## SWE 645 Homework / Assignment 4 – Extra Credit – Python Survey API

Author of document: Qasim Shahid

Date Due: 5/1/2025

Note: THIS DOCUMENT DOES NOT CONTAIN API TESTING. PLEASE WATCH THE VIDEO INCLUDED WITH THIS SUBMISSION TO SEE A FULL TEST USING POSTMAN. THIS DOCUMENT ONLY COVERS HOW TO SET UP THE PIPELINE AND AWS RESOURCES!

Demo video is contained in the zip file as “demo\_video.mp4”

This is my submission for the extra credit assignment, which essentially asks us to do the same thing from Assignment 3 where we created an API for survey data, but in Python rather than Java. All the steps you need to do to get the pipeline working will be included in this document. I will also go into some of the Python source code and provide explanation on design and implementation.

GitHub Repository: <https://github.com/qasimshahid/qshahid-swe645-asst4/>

Note: This repo is currently public but will be made private after this assignment is graded.

Another note: I ran out of budget on my AWS Learner Lab account, so this is being hosted on my own AWS account.

**For the below examples, please replace {rancher-ip} with the public IP address of the EC2 instance where your Rancher and Kubernetes cluster is being hosted.**

Website link:

- <http://{rancher-ip}:30008/api/version>
- (This is the link for an HTML page that shows the version of the app so that you can just see that it is up and running)

Other endpoints and what HTML method to use to access them

(Please replace any parameters in {} with your own specified parameters, such as ID of surveys)

- To get a survey by ID
  - GET
  - <http://{rancher-ip}:30008/api/surveys/{id}>
- To get all surveys
  - GET
  - <http://{rancher-ip}:30008/api/surveys>
- To delete a survey by ID
  - DELETE
  - <http://{rancher-ip}:30008/api/surveys/{id}>
- To post a new survey
  - POST (Must send a valid survey in HTML Request Body via JSON, please see the README.md of the GitHub repository for examples.)
  - <http://{rancher-ip}:30008/api/surveys>
- To update an existing survey by ID
  - PUT (Must send a valid survey in HTML Request Body via JSON, please see the README.md of the GitHub repository for examples.)
  - <http://{rancher-ip}:30008/api/surveys/{id}>

In this homework assignment, we are tasked to build a CI/CD pipeline using a very simple REST API for doing CRUD operations on survey data. We containerize the Python FastAPI application using Docker and then deploy it to a Kubernetes cluster, running at least 3 replicas. We use Rancher as a Kubernetes management platform to manage our cluster.

The Jenkins pipeline is triggered upon a push to the main branch of the GitHub repository. The pipeline does the following:

1. Get the most recent source code by cloning the repository
2. Copy our secret file for database configuration which we added to Jenkins to the working directory
3. Build the docker image defined in the Dockerfile using docker build. This image is based off of the base Python image.
4. Push the image we just built to Docker Hub (this requires an account, by default, the source code assumes mine is being used with username qshahid)
5. Deploys the latest container image for our app the Kubernetes cluster, creating a Deployment with 3 replicas (basically deploying the container to 3 pods). The app is exposed externally via a NodePort service, with port 30008 mapped to the container's port 8080, allowing external access by going to the public IP address of the EC2 instance where you are running Rancher.
  - a. Example: <http://35.153.204.96:30008/api/version>
  - b. 35.153.204.96 is the IP of the EC2 instance where my Rancher and Kubernetes cluster are running.

Please make sure you have a Docker Hub account, an AWS account, and a GitHub account. Please also clone/fork my GitHub repository to your own account so you can configure the webhook that allows us to automatically build and deploy whenever we push to the main branch. Please also make this GitHub repository public as that is what I used. Continue reading to see all the other things that are required to make this work.

## AWS Security Group Setup

Please duplicate / copy the following security group before doing anything since we will be assigning this security group to multiple things. This will make sure you are able to run without having port forwarding issues.

sg-0ec63124390edea9 - launch-wizard-1

Actions

Details

Security group name

launch-wizard-1

Security group ID

sg-0ec63124390edea9

Description

launch-wizard-1 created 2025-02-08T00:57:43.864Z

VPC ID

vpc-072de3089ca0547f5

Owner

654654395391

Inbound rules count

6 Permission entries

Outbound rules count

1 Permission entry

Inbound rules

Outbound rules

Sharing - new

VPC associations - new

Tags

Inbound rules (6)

Manage tags

Edit inbound rules

Q Search

|                          | Name | Security group rule ID | IP version | Type         | Protocol | Port range    | Source    | Description |
|--------------------------|------|------------------------|------------|--------------|----------|---------------|-----------|-------------|
| <input type="checkbox"/> | -    | sgr-0fb2e73e111ee9073  | IPv4       | HTTP         | TCP      | 80            | 0.0.0.0/0 | -           |
| <input type="checkbox"/> | -    | sgr-06aacf7b0720ac466  | IPv4       | Custom TCP   | TCP      | 30000 - 40000 | 0.0.0.0/0 | -           |
| <input type="checkbox"/> | -    | sgr-0bc2075d0806bf1a6  | IPv4       | HTTPS        | TCP      | 443           | 0.0.0.0/0 | -           |
| <input type="checkbox"/> | -    | sgr-01592b24772372719  | IPv4       | SSH          | TCP      | 22            | 0.0.0.0/0 | -           |
| <input type="checkbox"/> | -    | sgr-0329fb4a1bd2abcb5  | IPv4       | MySQL/Aurora | TCP      | 3306          | 0.0.0.0/0 | -           |
| <input type="checkbox"/> | -    | sgr-0378d456afb7ba97e  | IPv4       | Custom TCP   | TCP      | 8080          | 0.0.0.0/0 | -           |

## AWS Database Setup

Since we need to store our data somewhere, we need to create a free-tier MySQL database (or Amazon RDS) database. We need to make it publicly available so that our API can access it and we also need to create some credentials that we will remember, which will also be used in the deployment process so we can access the database.

Please also install MySQL Workbench on your computer so you can validate API operations:  
<https://www.mysql.com/products/workbench/>

Go to Amazon RDS and create a new database. Copy these settings:

### Use MySQL

**Create database** [Info](#)

**Choose a database creation method**

☒ **Standard create**  
You set all of the configuration options, including ones for availability, security, backups, and maintenance.

☐ **Easy create**  
Use recommended best-practice configurations. Some configuration options can be changed after the database is created.

**Engine options**

**Engine type** [Info](#)

☐ Aurora (MySQL Compatible) 

☐ Aurora (PostgreSQL Compatible) 

☒ MySQL 

☐ PostgreSQL 

☐ MariaDB 

☐ Oracle 

☐ Microsoft SQL Server 

☐ IBM Db2 

**Edition**  
☒ MySQL Community

**Engine version** [Info](#)  
View the engine versions that support the following database features.

**▼ Hide filters**

☒ **Show only versions that support the Multi-AZ DB cluster** [Info](#)  
Create a Multi-AZ DB cluster with one primary DB instance and two readable standby DB instances. Multi-AZ DB clusters provide up to 2x faster transaction commit latency and automatic failover in typically under 35 seconds.

☒ **Show only versions that support the Amazon RDS Optimized Writes** [Info](#)  
Amazon RDS Optimized Writes improves write throughput by up to 2x at no additional cost.

**Engine version**  
MySQL 8.0.40

☐ **Enable RDS Extended Support** [Info](#)  
Amazon RDS Extended Support is a paid offering [\[?\]](#). By selecting this option, you consent to being charged for this offering if you are running your database major version past the RDS end of standard support date for that version. Check the end of standard support date for your major version in the [RDS for MySQL documentation](#) [\[?\]](#).

Next, make sure to choose the free tier DB. Make a self-managed username and password. Note this down since we will need this to connect.

☐ Production  
Use defaults for high availability and fast, consistent performance.

☐ Dev/Test  
This instance is intended for development use outside of a production environment.

☒ Free Tier  
Use RDS Free Tier to develop new applications, test existing applications, or gain hands-on experience with Amazon RDS. [Info](#)

### Availability and durability

**Deployment options** [Info](#)  
Choose the deployment option that provides the availability and durability needed for your use case. AWS is committed to a certain level of uptime depending on the deployment option you choose. Learn more in the [Amazon RDS service level agreement \(SLA\)](#).

☐ Multi-AZ DB cluster deployment (3 instances)  
Creates a primary DB instance with two readable standbys in separate Availability Zones. This setup provides:  
• 99.99% uptime  
• Redundancy across Availability Zones  
• Increased read capacity  
• Reduced write latency

☐ Multi-AZ DB instance deployment (2 instances)  
Creates a primary DB instance with a non-readable standby instance in a separate Availability Zone. This setup provides:  
• 99.99% uptime  
• Redundancy across Availability Zones

☒ Single-AZ DB instance deployment (1 instance)  
Creates a single DB instance without standby instances. This setup provides:  
• 99.9% uptime  
• No data redundancy

**Write/read endpoint**  
AZ: 1  
Primary instance + SSD

**Reader endpoints**  
AZ: 2  
Readable standby + SSD  
AZ: 3  
Readable standby + SSD

**Write/read endpoint**  
AZ: 1  
Primary instance

**Standby (no endpoint)**  
AZ: 2  
Standby

**Write/read endpoint**  
AZ: 1  
Primary instance

### Settings

**DB instance identifier** [Info](#)  
Type a name for your DB instance. The name must be unique across all DB instances owned by your AWS account in the current AWS Region.  
  
The DB instance identifier is case-insensitive, but is stored as all lowercase (as in "mydbinstance"). Constraints: 1 to 63 alphanumeric characters or hyphens. First character must be a letter. Can't contain two consecutive hyphens. Can't end with a hyphen.

**▼ Credentials Settings**

**Master username** [Info](#)  
Type a login ID for the master user of your DB instance.  
  
1 to 16 alphanumeric characters. The first character must be a letter.

**Credentials management**  
You can use AWS Secrets Manager or manage your master user credentials.  
☐ Managed in AWS Secrets Manager - most secure  
RDS generates a password for you and manages its lifecycle using AWS Secrets Manager.

☒ Self managed  
Create your own password or have RDS create a password that you manage.

☐ Auto generate password  
Amazon RDS can generate a password for you, or you can specify your own password.

**Master password** [Info](#)  
  
**Password strength** [Info](#) Very weak  
Minimum constraints: At least 8 printable ASCII characters. Can't contain any of the following symbols: / " @

Then, there will be a bunch of options. Scroll down to connectivity and copy the following:

**Connectivity** [Info](#)

**Compute resource** [Info](#)  
Choose whether to set up a connection to a compute resource for this database. Setting up a connection will automatically change connectivity settings so that the compute resource can connect to this database.

☒ Don't connect to an EC2 compute resource  
Don't set up a connection to a compute resource for this database. You can manually set up a connection to a compute resource later.

☐ Connect to an EC2 compute resource  
Set up a connection to an EC2 compute resource for this database.

**Network type** [Info](#)  
To use dual-stack mode, make sure that you associate an IPv6 CIDR block with a subnet in the VPC you specify.

☒ IPv4  
Your resources can communicate only over the IPv4 addressing protocol.

☐ Dual-stack mode  
Your resources can communicate over IPv4, IPv6, or both.

**Virtual private cloud (VPC)** [Info](#)  
Choose the VPC. The VPC defines the virtual networking environment for this DB instance.  
  
6 Subnets, 6 Availability Zones  
Only VPCs with a corresponding DB subnet group are listed.

**DB subnet group** [Info](#)  
Choose the DB subnet group. The DB subnet group defines which subnets and IP ranges the DB instance can use in the VPC that you selected.  
  
6 Subnets, 6 Availability Zones

**Public access** [Info](#)  
☒ Yes  
RDS assigns a public IP address to the database. Amazon EC2 instances and other resources outside of the VPC can connect to your database. Resources inside the VPC can also connect to the database. Choose one or more VPC security groups that specify which resources can connect to the database.

☐ No  
RDS doesn't assign a public IP address to the database. Only Amazon EC2 instances and other resources inside the VPC can connect to your database. Choose one or more VPC security groups that specify which resources can connect to the database.

**VPC security group (firewall)** [Info](#)  
Choose one or more VPC security groups to allow access to your database. Make sure that the security group rules allow the appropriate incoming traffic.

☒ Choose existing  
Choose existing VPC security groups

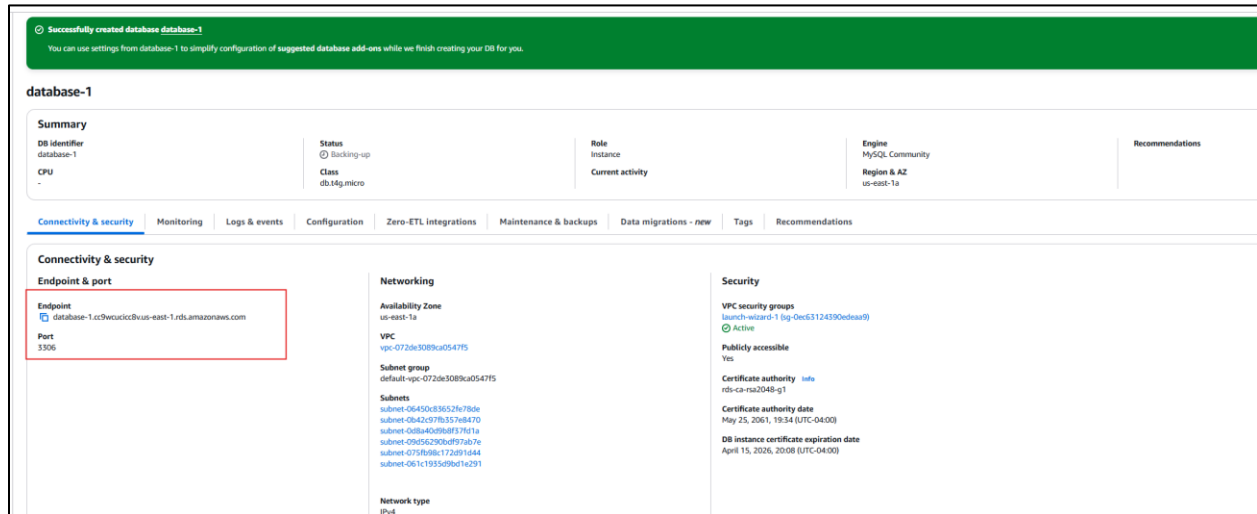
☐ Create new  
Create new VPC security group

**Existing VPC security groups**

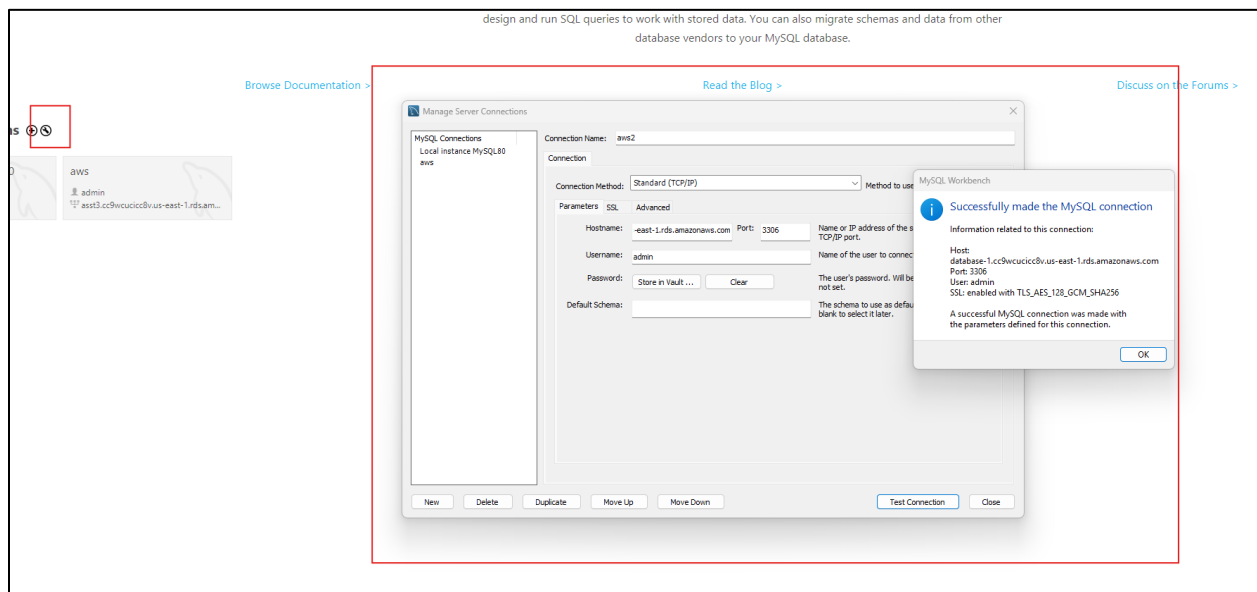
**Availability Zone** [Info](#)

After this, click “Create database”

Now, open MySQL Workbench. Try to connect to your database using the IP, 3306 port, and your username and password that you set.

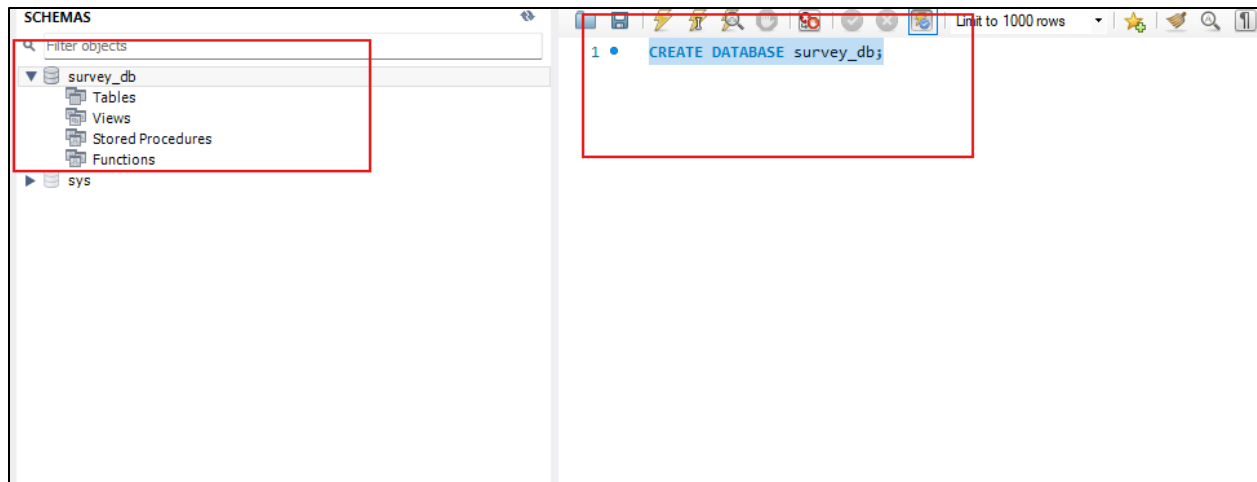


You should see this if you did everything correctly. Create the connection and connect to the DB. We need to execute one command in it.



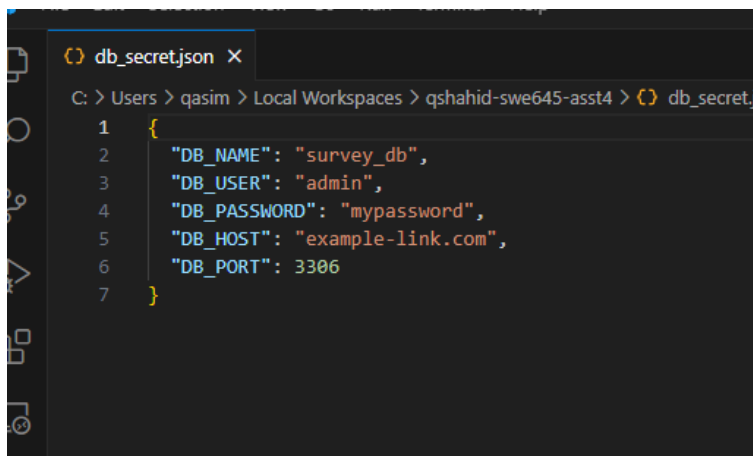
Once connected, please go to the GitHub repository and go to the createDb.sql file and copy the command to create the database. This is the only SQL command we need to run.

“CREATE DATABASE survey\_db;”



Now, you are done with database setup. Note down the AWS DB link and your credentials. We will be injecting these credentials into our Jenkins instance by creating a new credential that uses a secret file. This will be “db\_secret.json” so that our API can connect to the database.

Create a file exactly like this but use your own DB link and credentials. Note that I specify the database I want to connect to, survey\_db, and you should do this as well. We created this database in the previous step.



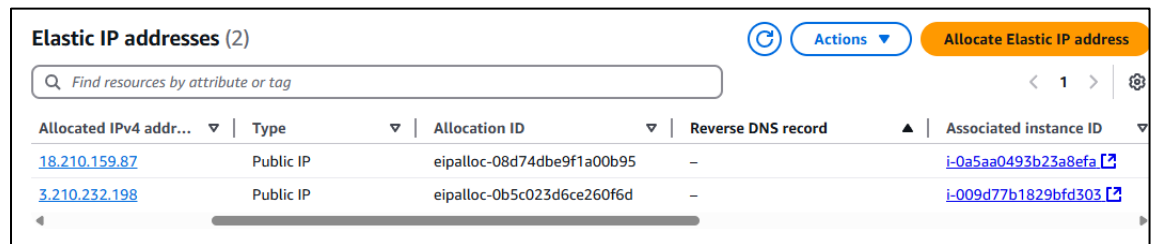
Have it be the same name as well, “db\_secret.json”. Like I said, we will give this file to Jenkins, which will copy this into the working directory and use it. We want to keep these details away from the public which is why we have to add it to Jenkins ourselves. The one in the GitHub repository, also named “db\_secret.json”, is just an example and doesn’t actually contain any sensitive information. You have to replace this with your own DB credentials and link to get it to work.

Once the file has been created, save it somewhere where you will remember.

## AWS EC2 Setup

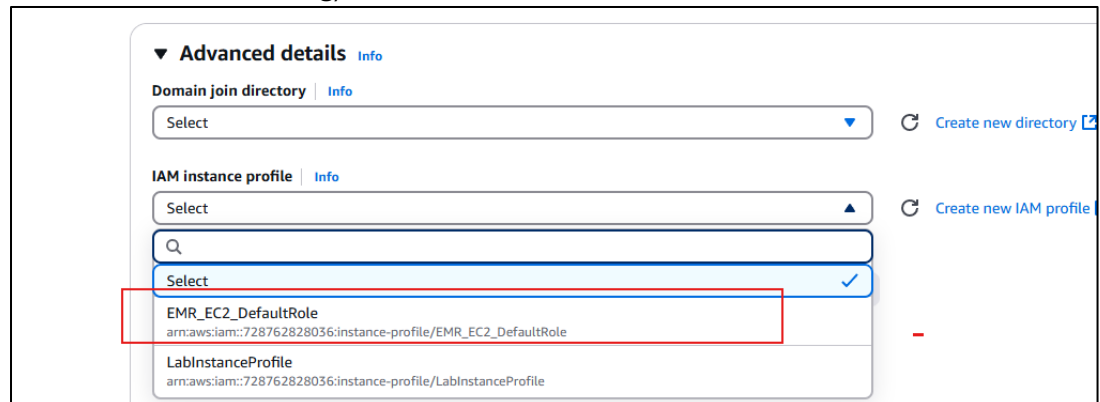
This assignment makes extensive use of AWS. Please keep that in mind since this might cost you money.

- 2 EC2 machines:
  - 1 t2.large machine, which will host Rancher
  - 1 t2.large machine, which will host Jenkins
  - Please use Ubuntu on all of these machines and configure them to have 30 GB of storage.
  - From this point on, I will refer to them as the Rancher instance and the Jenkins instance.
- Elastic IPs
  - Please attach an elastic IP to both of these machines as it makes it easier to work with.



| Allocated IPv4 address        | Type      | Allocation ID              | Reverse DNS record | Associated instance ID              |
|-------------------------------|-----------|----------------------------|--------------------|-------------------------------------|
| <a href="#">18.210.159.87</a> | Public IP | eipalloc-08d74dbe9f1a00b95 | -                  | <a href="#">i-0a5aa0493b23a8efa</a> |
| <a href="#">3.210.232.198</a> | Public IP | eipalloc-0b5c023d6ce260f6d | -                  | <a href="#">i-009d77b1829bfd303</a> |

- Please add this setting in “Advanced” when creating your instances (if you are using AWS Learner Lab, which I am not using):



▼ **Advanced details** [Info](#)

Domain join directory [Info](#)  
Select [Create new directory](#)

IAM instance profile [Info](#)  
Select [Create new IAM profile](#)

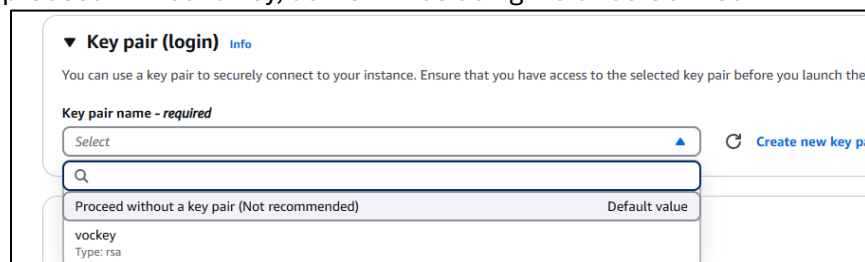
Search:

Select ☒

EMR\_EC2\_DefaultRole  
arn:aws:iam::728762828036:instance-profile/EMR\_EC2\_DefaultRole

LabInstanceProfile  
arn:aws:iam::728762828036:instance-profile/LabInstanceProfile

- Please proceed without a key, as we will be using Instance Connect.



▼ **Key pair (login)** [Info](#)

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - *required*  
Select [Create new key pair](#)

Search:

Proceed without a key pair (Not recommended) ☒ Default value

vockey  
Type: rsa



- Please create the following security group and apply it to both the Rancher instance and the Jenkins instance.

**sg-0ec63124390edea9 - launch-wizard-1** Actions

**Details**

|                                               |                                                    |                                                                        |                                        |
|-----------------------------------------------|----------------------------------------------------|------------------------------------------------------------------------|----------------------------------------|
| <b>Security group name</b><br>launch-wizard-1 | <b>Security group ID</b><br>sg-0ec63124390edea9    | <b>Description</b><br>launch-wizard-1 created 2025-02-08T00:57:43.864Z | <b>VPC ID</b><br>vpc-072de3089ca0547f5 |
| <b>Owner</b><br>654654395391                  | <b>Inbound rules count</b><br>6 Permission entries | <b>Outbound rules count</b><br>1 Permission entry                      |                                        |

**Inbound rules** | Outbound rules | Sharing - new | VPC associations - new | Tags

**Inbound rules (6)** Manage tags Edit inbound rules

| <input type="checkbox"/> | Name | Security group rule ID | IP version | Type         | Protocol | Port range    | Source    | Description |
|--------------------------|------|------------------------|------------|--------------|----------|---------------|-----------|-------------|
| <input type="checkbox"/> | -    | sg-0fb2e73e111ee9073   | IPv4       | HTTP         | TCP      | 80            | 0.0.0.0/0 | -           |
| <input type="checkbox"/> | -    | sg-05aac7b0720ac466    | IPv4       | Custom TCP   | TCP      | 30000 - 40000 | 0.0.0.0/0 | -           |
| <input type="checkbox"/> | -    | sg-0bc2075d0806bf1a6   | IPv4       | HTTPS        | TCP      | 443           | 0.0.0.0/0 | -           |
| <input type="checkbox"/> | -    | sg-01592b24772372719   | IPv4       | SSH          | TCP      | 22            | 0.0.0.0/0 | -           |
| <input type="checkbox"/> | -    | sg-0329fb4a1bd2abcb5   | IPv4       | MySQL/Aurora | TCP      | 3306          | 0.0.0.0/0 | -           |
| <input type="checkbox"/> | -    | sg-0378d456afb7ba97e   | IPv4       | Custom TCP   | TCP      | 8080          | 0.0.0.0/0 | -           |

- 
- Leave outbound rules as the default.

- After you do all this, you should be ready to proceed to the next step of setting up your EC2 machines. In particular, we need to install some dependencies.

For both machines, you need to execute the following commands. These will install some dependencies, such as Docker, kubectl, and generally just update the machines.

```
sudo su
sudo apt-get update -y
sudo apt-get upgrade -y

curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh

sudo snap install kubectl --classic
```

## Setup Rancher Instance and Cluster

In order to get Rancher on your t2.large instance, do the following.  
Login to the instance using instance connect:

EC2 Instance Connect | Session Manager | SSH client | EC2 serial console

Instance ID  
i-05649b9f3d5a98b81 (ex)

Connection Type

☒ Connect using EC2 Instance Connect  
Connect using the EC2 Instance Connect browser-based client, with a public IPv4 or IPv6 address.

☐ Connect using EC2 Instance Connect Endpoint  
Connect using the EC2 Instance Connect browser-based client, with a private IPv4 address and a VPC endpoint.

☒ Public IPv4 address  
52.55.20.9

☐ IPv6 address  
-

Username  
Enter the username defined in the AMI used to launch the instance. If you didn't define a custom username, use the default username, ubuntu.

ubuntu

Note: In most cases, the default username, ubuntu, is correct. However, read your AMI usage instructions to check if the AMI owner has changed the default AMI username.

Cancel Connect

Run all the commands mentioned at the end of the previous section.

```
sudo su
sudo apt-get update -y
sudo apt-get upgrade -y

curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh

sudo snap install kubectl --classic
```

Then, to use Rancher, run the following command:

```
sudo docker run --privileged -d --restart=unless-stopped -p 80:80 -p 443:443
rancher/rancher
```

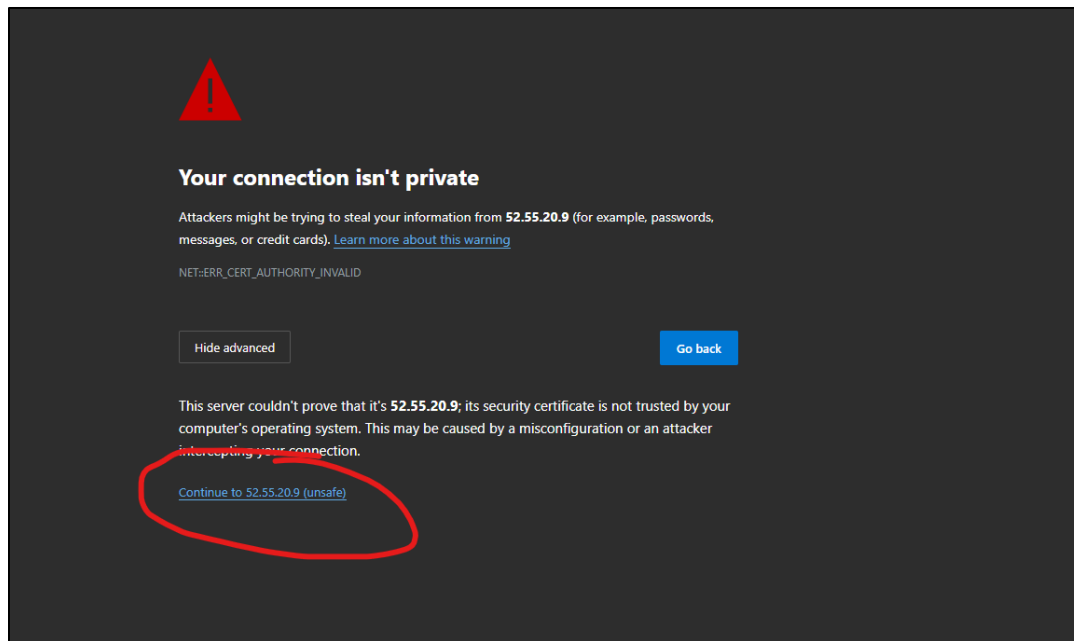
After that, go to the following link to access Rancher:

`http://{rancher-ec2-public-ip}:80`

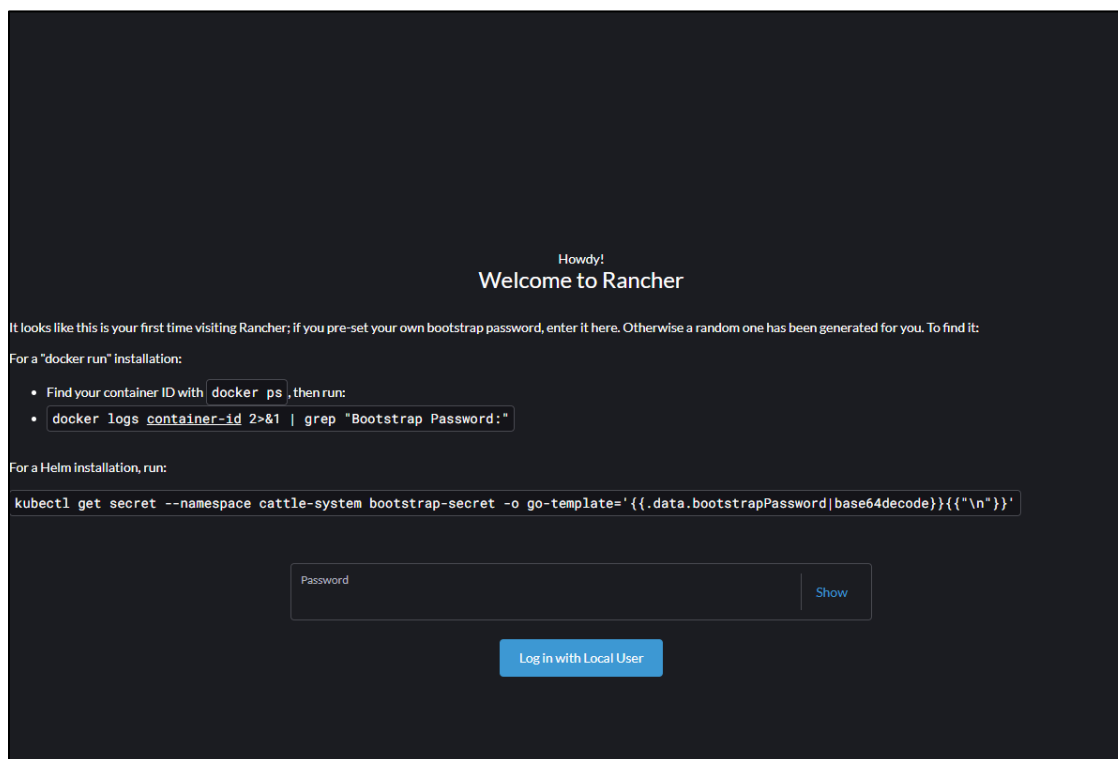
For example,

`http://52.55.20.9:80`

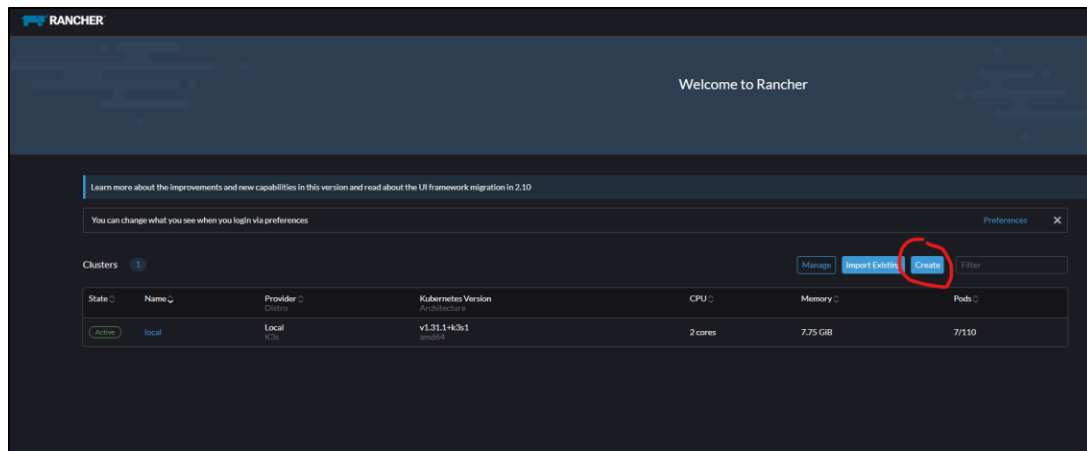
It will tell you the site is not secure, but just proceed past the warnings.



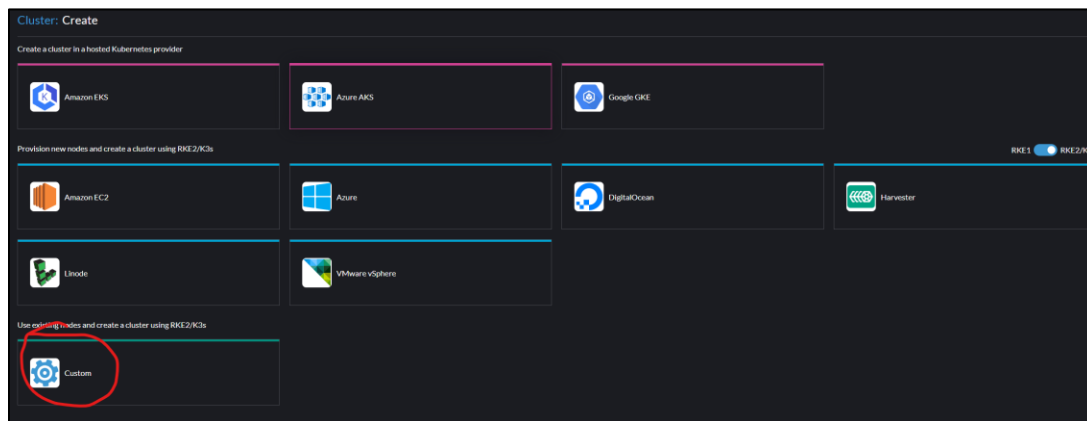
After that, you will be met by this UI and login page from Rancher.



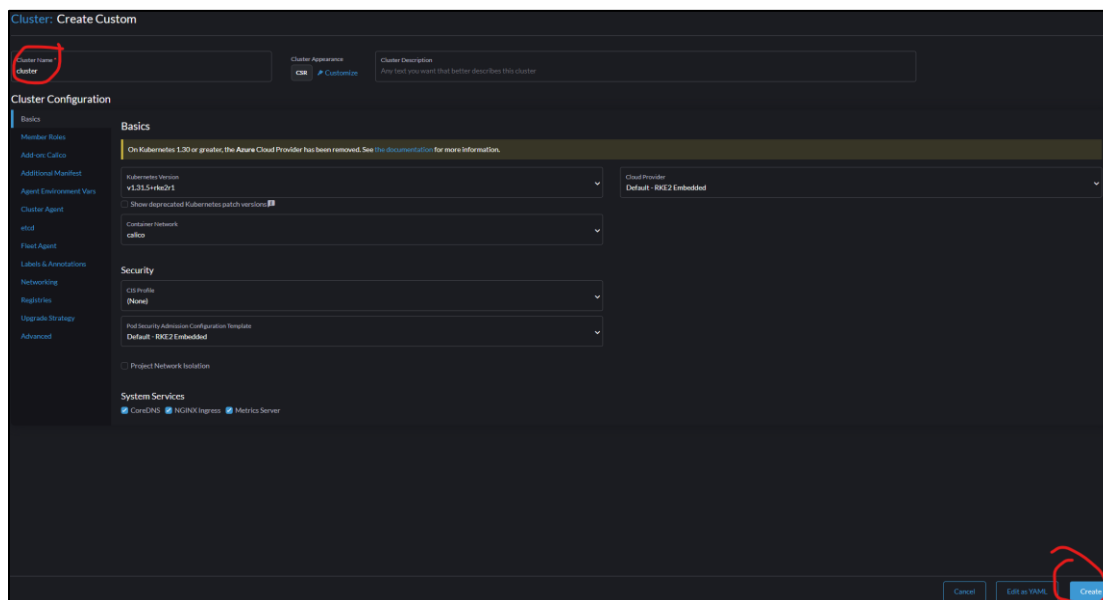
Follow the instructions and login to Rancher with the bootstrap password. Create your own password so that you remember the login. Follow the sign-up instructions and login to Rancher.



Let's create our new cluster. Use RKE2/3 to create a custom cluster.



You don't need to do much. Just name it "cluster" and press create.



Cluster: cluster Updating

Namespace: fleet-default Age: Just now

waiting for at least one control plane, etcd, and worker node to be registered

Provisioner: RKE2

Machines Provisioning Log **Registration** Snapshots Conditions Recent Events Related Resources

**Step 1**

**Node Role**

Choose what roles the node will have in the cluster. The cluster needs to have at least one node with each role.

☒ etcd ☒ Control Plane ☒ Worker

Show Advanced

**Step 2**

**Registration Command**

Run this command on each of the existing Linux machines you want to register.

```
curl --insecure -fL https://54.80.152.86/system-agent-install.sh | sudo sh -s - --server https://54.80.152.86 --label 'cattle.io/os=linux' --token 4pgf8ppjlvqzs2r61jfg4bs2wbrzgff8k56vtv42kxrt5blsqczz --ca-checksum c3088f50a79f38f652b54775f722530547b644658d8e3c0caf3ef46e67cae9 --etcd --controlplane --worker
```

☒ Insecure: Select this to skip TLS verification if your server has a self-signed certificate.

Run this command in PowerShell on each of the existing Windows machines you want to register. Windows nodes can only be workers.

The cluster must be up and running with Linux etcd, control plane, and worker nodes before the registration command for adding Windows workers will display.

Then, you need to register a control plane, etcd, and worker node. You can do these on separate machines, but let's just do it here on the Rancher instance since this is a large machine and can handle it. Please select the "insecure" option of the registration command or this won't work. Copy it and run it on your Rancher instance.

```
root@ip-172-31-46-212:/home/ubuntu# curl --insecure -fL https://54.80.152.86/system-agent-install.sh | sudo sh -s - --server https://54.80.152.86 --label 'cattle.io/os=linux' --token 4pgf8ppjlvqzs2r61jfg4bs2wbrzgff8k56vtv42kxrt5blsqczz --ca-checksum c3088f50a79f38f652b54775f722530547b644658d8e3c0caf3ef46e67cae9 --etcd --controlplane --worker
% Total    % Received % Xferd  Average Speed   Time    Time     Current
                                 Dload  Upload   Total   Spent    Left     Speed
100 33662  0 33662    0     0  3446k      0 --:--:-- --:--:-- --:--:-- 3652k
[INFO] Label: cattle.io/os=linux
[INFO] Role requested: etcd
[INFO] Role requested: controlplane
[INFO] Role requested: worker
[INFO] CA strict verification is set to true
[INFO] Using default agent configuration directory /etc/rancher/agent
[INFO] Using default agent var directory /var/lib/rancher/agent
[INFO] Successfully downloaded CA certificate
[INFO] Value from https://54.80.152.86/cacerts is an x509 certificate
[INFO] Successfully tested Rancher connection
[INFO] Downloading rancher-system-agent binary from https://54.80.152.86/assets/rancher-system-agent-amd64
[INFO] Successfully downloaded the rancher-system-agent binary.
[INFO] Downloading rancher-system-agent-uninstall.sh script from https://54.80.152.86/assets/system-agent-uninstall.sh
[INFO] Successfully downloaded the rancher-system-agent-uninstall.sh script.
[INFO] Generating Cattle ID
[INFO] Successfully downloaded Rancher connection information
[INFO] systemd: Creating service file
[INFO] Creating environment file /etc/systemd/system/rancher-system-agent.env
[INFO] Enabling rancher-system-agent.service
Created symlink /etc/systemd/system/multi-user.target.wants/rancher-system-agent.service -> /etc/systemd/system/rancher-system-agent.service.
[INFO] Starting/restarting rancher-system-agent.service
root@ip-172-31-46-212:/home/ubuntu#
```

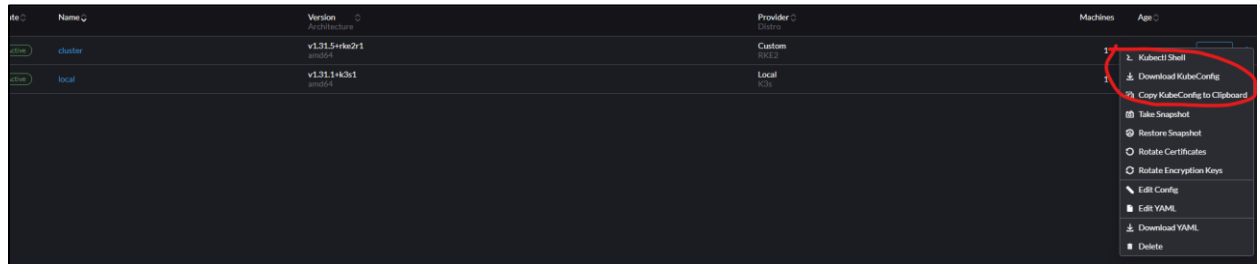
Wait about 5 minutes at this point for everything to be set up properly. During this time, Rancher will show that the cluster is updating. Once 5 minutes has elapsed, you should now see that your new Kubernetes cluster is active.

| <input type="checkbox"/> State             | Name    | Version<br>Architecture |
|--------------------------------------------|---------|-------------------------|
| <input checked="" type="checkbox"/> Active | cluster | v1.31.5+rke2r1<br>amd64 |
| <input checked="" type="checkbox"/> Active | local   | v1.31.1+k3s1<br>amd64   |

We have our K8s cluster created, named “cluster”, and it has 1 worker node, 1 control plane, and 1 etcd node, all on the same machine as where our cluster is.

The last step that you should do is download the kubeconfig file to your computer, as we will need it here shortly for the Jenkins instance:

This can be done from the “Cluster Management” tab on the left.



This allows us to connect to our cluster from different machines and instances (like our Jenkins instance that we will make after this) using the kubeconfig file.

Congratulations, at this point, you are done with everything regarding the Rancher instance. Rancher has been setup, and you can access it using the elastic IP we assigned to the instance at `http://{rancher-ec2-public-ip}:80`

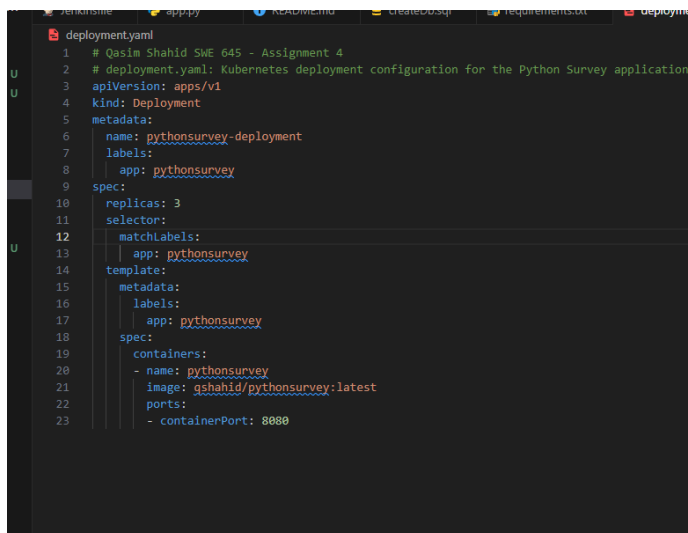
## GitHub Repository Setup

GitHub Repository: <https://github.com/qasimshahid/qshahid-swe645-asst4/>

Please fork this repository or copy all the files from it and add them to a new repository that you created. This has pretty much everything you need, but you do need to make some changes.

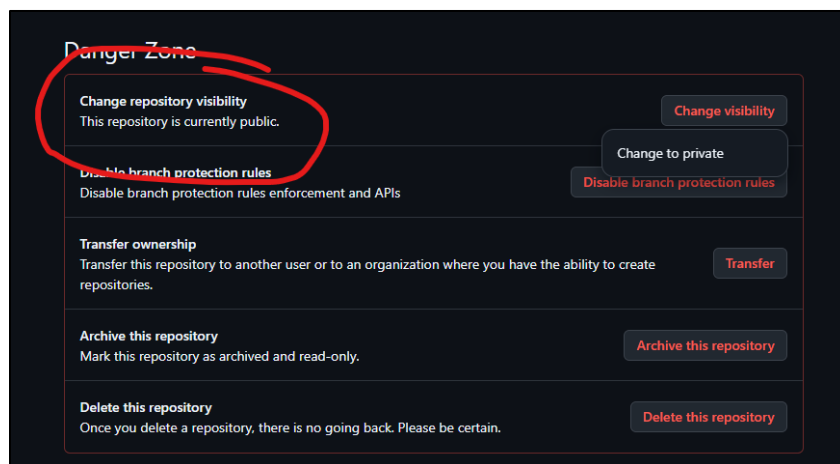
First of all, please fix these settings to be your own. They should utilize the link of your GitHub repository and whatever your username is for Docker Hub and what your image should be called.

For example, here you can see I am using my Docker Hub username. Replace this with your own. Please look through all the files of the GitHub repository and replace it wherever applicable. You need to make sure it is all configured correctly for our pipeline to work without access issues.



```
1 # Qasim Shahid SWE 645 - Assignment 4
2 # deployment.yaml: Kubernetes deployment configuration for the Python Survey application.
3 apiVersion: apps/v1
4 kind: Deployment
5 metadata:
6   name: pythonsurvey-deployment
7   labels:
8     app: pythonsurvey
9 spec:
10   replicas: 3
11   selector:
12     matchLabels:
13       app: pythonsurvey
14   template:
15     metadata:
16       labels:
17         app: pythonsurvey
18     spec:
19       containers:
20       - name: pythonsurvey
21         image: qshahid/pythonsurvey:latest
22         ports:
23         - containerPort: 8080
```

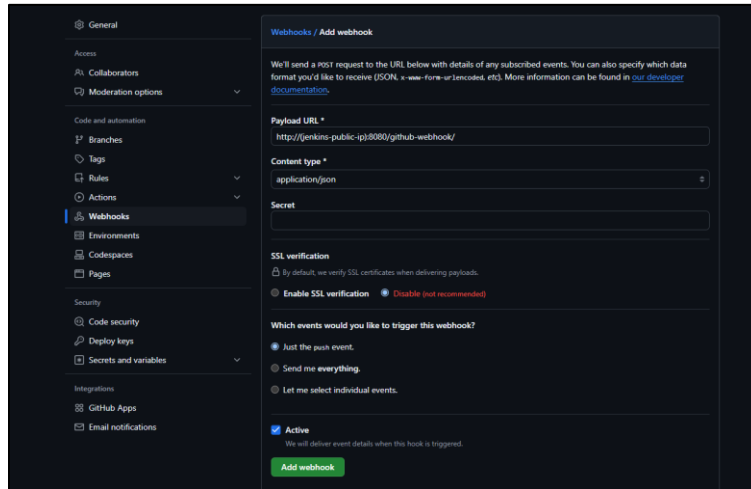
After you do all that, make sure the GitHub repository is public by going into the repository settings:



Please also at this point go into the webhook settings and create a new webhook that points to the following link:

`http://{jenkins-ec2-public-ip}:8080/github-webhook/`

Replace `{jenkins-ec2-public-ip}` with the public IP of your Jenkins instance and use the following settings:

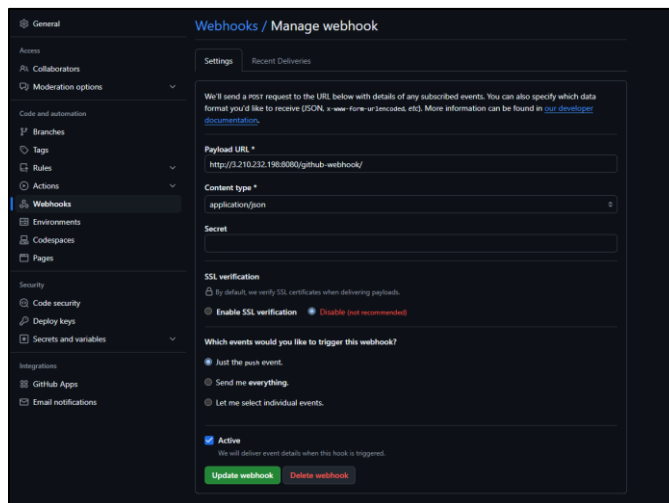


The screenshot shows the Jenkins 'Webhooks / Add webhook' configuration page. The left sidebar contains a navigation menu with categories like 'General', 'Access', 'Code and automation', 'Security', and 'Integrations'. The 'Webhooks' option is selected. The main content area is titled 'Webhooks / Add webhook' and contains the following fields and options:

- Payload URL \***: `http://jenkins-public-ip:8080/github-webhook/`
- Content type \***: `application/json`
- Secret**: (empty text field)
- SSL verification**: A section with a warning icon and text 'By default, we verify SSL certificates when delivering payloads.' Below it are two radio buttons: 'Enable SSL verification' (selected) and 'Disable (not recommended)'.
- Which events would you like to trigger this webhook?**: Three radio buttons: 'Just the push event.' (selected), 'Send me everything.', and 'Let me select individual events.'
- Active**: A checked checkbox with the text 'We will deliver event details when this hook is triggered.'
- Add webhook**: A green button at the bottom.

Add this webhook and disable SSL verification.

Example:



The screenshot shows the Jenkins 'Webhooks / Manage webhook' configuration page. The left sidebar is the same as in the previous image. The main content area is titled 'Webhooks / Manage webhook' and has two tabs: 'Settings' (selected) and 'Recent Deliveries'. The 'Settings' tab contains the following fields and options:

- Payload URL \***: `http://3.210.232.198:8080/github-webhook/`
- Content type \***: `application/json`
- Secret**: (empty text field)
- SSL verification**: A section with a warning icon and text 'By default, we verify SSL certificates when delivering payloads.' Below it are two radio buttons: 'Enable SSL verification' and 'Disable (not recommended)' (selected).
- Which events would you like to trigger this webhook?**: Three radio buttons: 'Just the push event.' (selected), 'Send me everything.', and 'Let me select individual events.'
- Active**: A checked checkbox with the text 'We will deliver event details when this hook is triggered.'
- Update webhook**: A green button.
- Delete webhook**: A red button.

This webhook will allow us to automatically build and deploy our application whenever we push to main branch on our GitHub repository. GitHub will recognize our push event and send a POST request to our Jenkins server, which will then kick off the pipeline.

Congratulations, at this point, you are done with setting up your GitHub repo. Push all the code changes and modifications you made and save all of these setting.



## Jenkins Instance Setup

So, this is the other EC2 machine we created at the start, and this is where we will host our Jenkins server. This will allow us to automate our builds and make a pipeline that gets triggered whenever we push to the main branch of our GitHub repository.

Please connect to the instance using Instance Connect:

The screenshot shows the AWS Management Console interface for connecting to an EC2 instance. The 'EC2 Instance Connect' tab is active. The instance ID is 'i-0910c6ae322c31abd (jenk)'. Under 'Connection Type', the 'Connect using EC2 Instance Connect' option is selected. The 'Public IPv4 address' is 54.87.102.0. The 'Username' is 'ubuntu'. A note at the bottom states: 'Note: In most cases, the default username, ubuntu, is correct. However, read your AMI usage instructions to check if the AMI owner has changed the default AMI username.' Buttons for 'Cancel' and 'Connect' are at the bottom right.

After that, please execute these commands if you haven't already. These commands are required for both instances:

```
sudo su
sudo apt-get update -y
sudo apt-get upgrade -y

curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh

sudo snap install kubectl --classic
```

After that, please execute the following commands or follow the instructions from the official Jenkins site on how to download for Ubuntu (<https://www.jenkins.io/doc/book/installing/linux/>):

```
sudo apt install fontconfig openjdk-17-jre -y

sudo wget -O /usr/share/keyrings/jenkins-keyring.asc \
https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc]" \
https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
/etc/apt/sources.list.d/jenkins.list > /dev/null
sudo apt-get update
sudo apt-get install jenkins -y
```

```
sudo usermod -a -G docker jenkins
```

(This command is SUPER IMPORTANT, make sure to execute this one or else builds might fail. If it does not work after your first build and you get some docker permission issues, execute it again)

```
sudo systemctl enable jenkins
sudo systemctl start jenkins
sudo systemctl status jenkins
```

These commands will install Java and Jenkins. They will also make Docker accessible to Jenkins without needing to do sudo. The last command will also print a password, please store this as we will need it here in a second:

```

root@ip-172-31-91-46:/home/ubuntu# ls
get-docker.sh
root@ip-172-31-91-46:/home/ubuntu# sudo usermod -s /usr/sbin/docker jenkins
root@ip-172-31-91-46:/home/ubuntu# sudo systemctl enable jenkins
Synchronizing state of jenkins.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable jenkins
root@ip-172-31-91-46:/home/ubuntu# sudo systemctl start jenkins
root@ip-172-31-91-46:/home/ubuntu# sudo systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
   Active: active (running) since Mon 2025-03-10 11:28:15 UTC; 26s ago
     Main PID: 3172 (java)
       Tasks: 43 (limit: 1129)
      Memory: 274.5M (peak: 330.0M)
         CPU: 15.35s
    CGroup: /system.slice/jenkins.service
            └─3172 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/.../bin/mar --webroot=/var/cache/jenkins/war --httpPort=8080

Mar 10 11:28:09 ip-172-31-91-46 jenkins[3172]: 2025-03-10 11:28:09.343Z [id=33] INFO hudson.lifecycle.Lifecycle#start: This may also be found at /var/lib/jenkins/secrets/InitialAdminPassword
Mar 10 11:28:09 ip-172-31-91-46 jenkins[3172]: 2025-03-10 11:28:09.343Z [id=33] INFO hudson.lifecycle.Lifecycle#start: This may also be found at /var/lib/jenkins/secrets/InitialAdminPassword
Mar 10 11:28:09 ip-172-31-91-46 jenkins[3172]: 2025-03-10 11:28:09.343Z [id=33] INFO hudson.lifecycle.Lifecycle#start: This may also be found at /var/lib/jenkins/secrets/InitialAdminPassword
Mar 10 11:28:09 ip-172-31-91-46 jenkins[3172]: 2025-03-10 11:28:09.343Z [id=33] INFO hudson.lifecycle.Lifecycle#start: This may also be found at /var/lib/jenkins/secrets/InitialAdminPassword
Mar 10 11:28:09 ip-172-31-91-46 jenkins[3172]: 2025-03-10 11:28:09.343Z [id=33] INFO hudson.lifecycle.Lifecycle#start: This may also be found at /var/lib/jenkins/secrets/InitialAdminPassword
Mar 10 11:28:15 ip-172-31-91-46 jenkins[3172]: 2025-03-10 11:28:15.211+0000 [id=34] INFO hudson.InitReactorRunner$1#onAttained: Completed initialization
Mar 10 11:28:15 ip-172-31-91-46 jenkins[3172]: 2025-03-10 11:28:15.240+0000 [id=23] INFO hudson.lifecycle.Lifecycle#onReady: Jenkins is fully up and running
Mar 10 11:28:15 systemd[1]: Started jenkins.service - Jenkins Continuous Integration Server
Mar 10 11:28:15 ip-172-31-91-46 jenkins[3172]: 2025-03-10 11:28:15.531+0000 [id=35] INFO hudson.DownloadServicesDownloadable#load: Obtained the updated data file for hudson.tasks.Maven.MavenInstaller
Mar 10 11:28:15 ip-172-31-91-46 jenkins[3172]: 2025-03-10 11:28:15.534+0000 [id=46] INFO hudson.util.Retrier$Start: Performed the action check updates server successfully at the attempt #1
root@ip-172-31-91-46:/home/ubuntu#

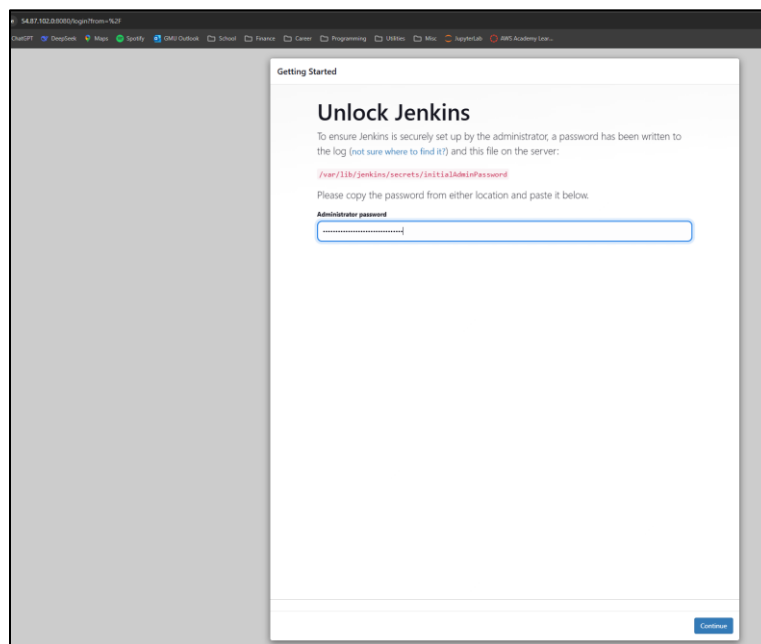
```

Now, the server is up. To go to it, use the elastic public IP we created at the start and go to this link:

http://{jenkins-ec2-public-ip}:8080

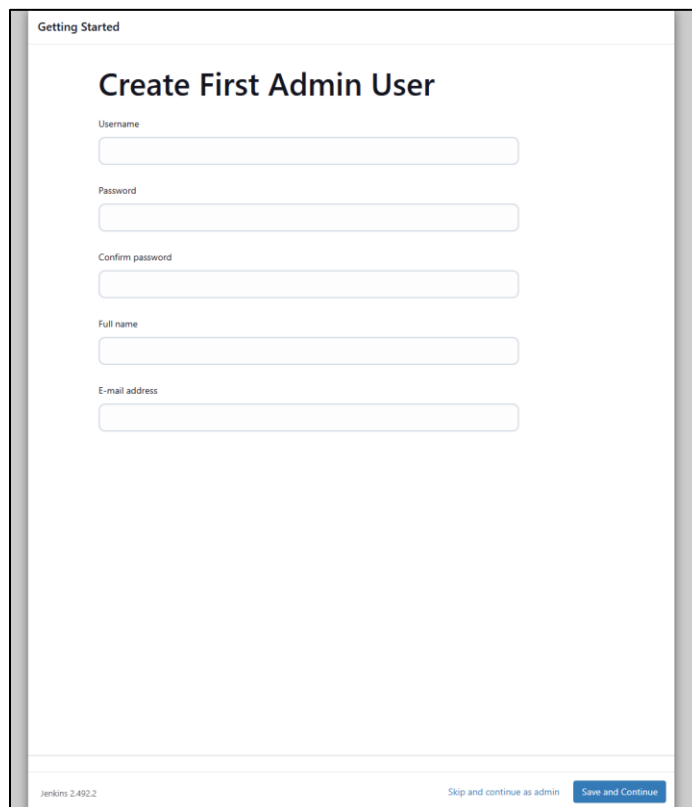
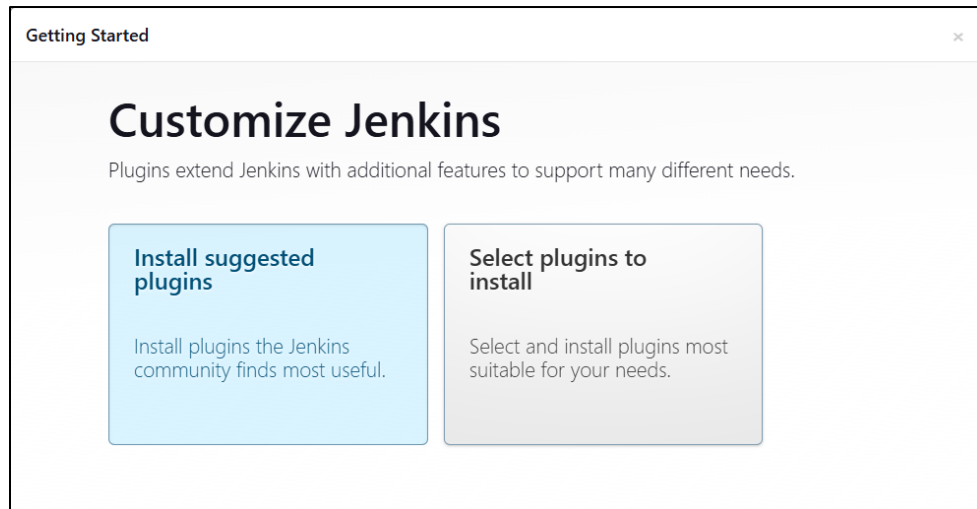
For example:

<http://54.87.102.0:8080/>



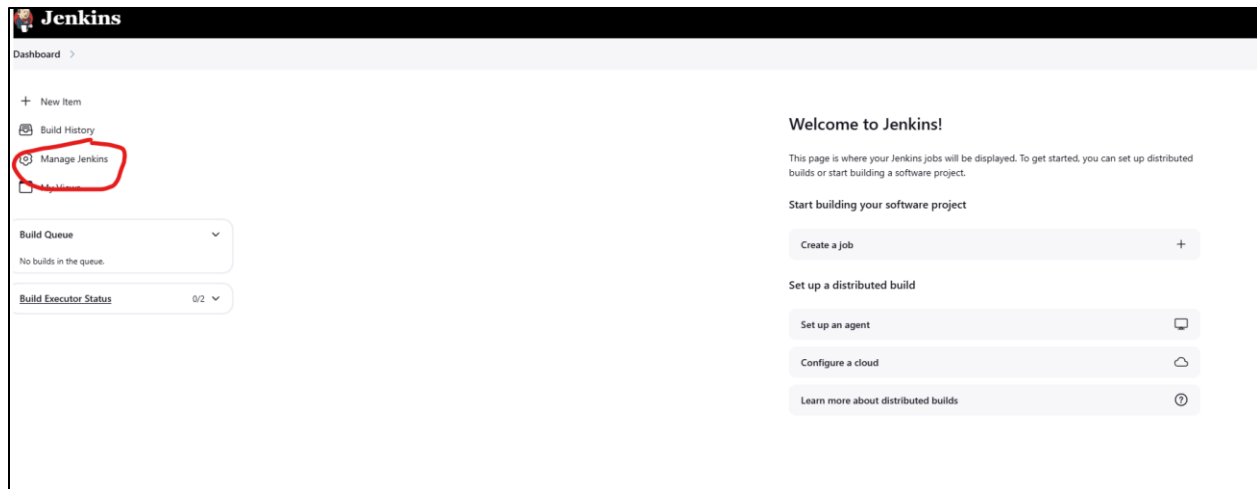
Go the link and Jenkins will ask you for the password from earlier. Paste it in.

Jenkins will now ask you what plugins you want. Select “Install suggested plugins.” We will add some more later on.

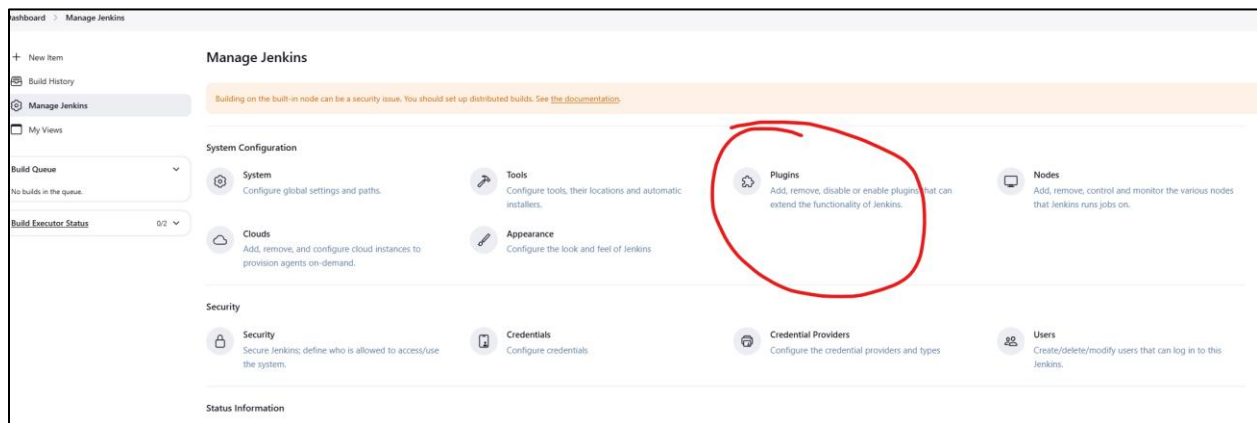
A screenshot of the Jenkins 'Getting Started' window, specifically the 'Create First Admin User' form. The title 'Create First Admin User' is at the top. Below it are five input fields: 'Username', 'Password', 'Confirm password', 'Full name', and 'E-mail address'. At the bottom of the form, there is a footer area containing the text 'Jenkins 2.402.2', a link 'Skip and continue as admin', and a blue button labeled 'Save and Continue'.

Then, create a username and password. Go through the sign-up process. After this, keep clicking continue until the setup is complete.

We are now done with the initial setup. You should be on this screen:



Now, go to “Manage Jenkins” so that we can install some plugins.



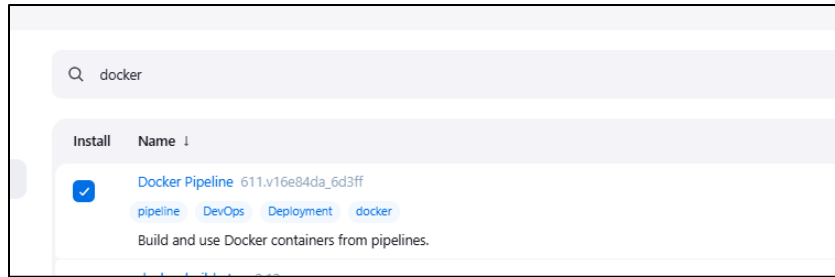
We need to install three plugins:

Docker plugin

Kubernetes CLI plugin

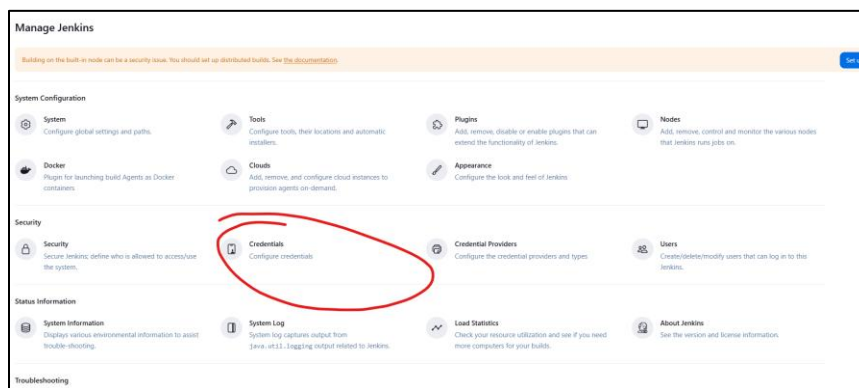
Docker Pipeline plugin





Install these and wait for them to finish downloading. We need these to build our image and deploy to our cluster.

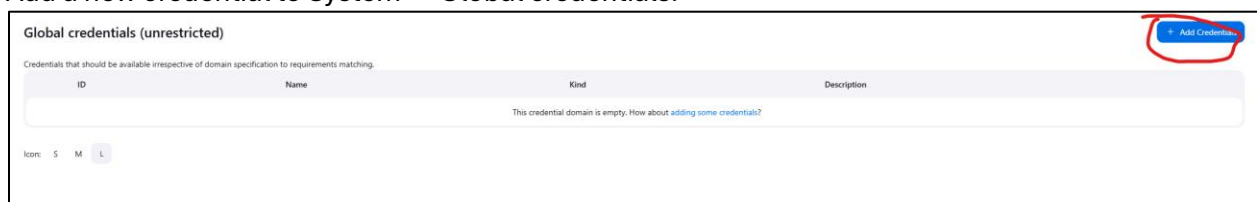
After these are complete, go back to “Manage Jenkins” but this time, go to Credentials:



We will now add our Docker Hub credentials and the kubeconfig file that you should have saved from the Rancher Instance setup.

First, the kubeconfig file:

Add a new credential to System -> Global credentials.



Make it a “Secret file”

Copy these settings:

ID - kubeconfig\_credentials

File – Select the .yaml file we downloaded earlier when setting up the Rancher Instance. This was the KubeConfig file of our cluster that we downloaded from Rancher.

Kind

Secret file

Scope ?

Global (Jenkins, nodes, items, all child items, etc)

File

Choose File cluster.yaml

ID ?

kubeconfig\_credentials

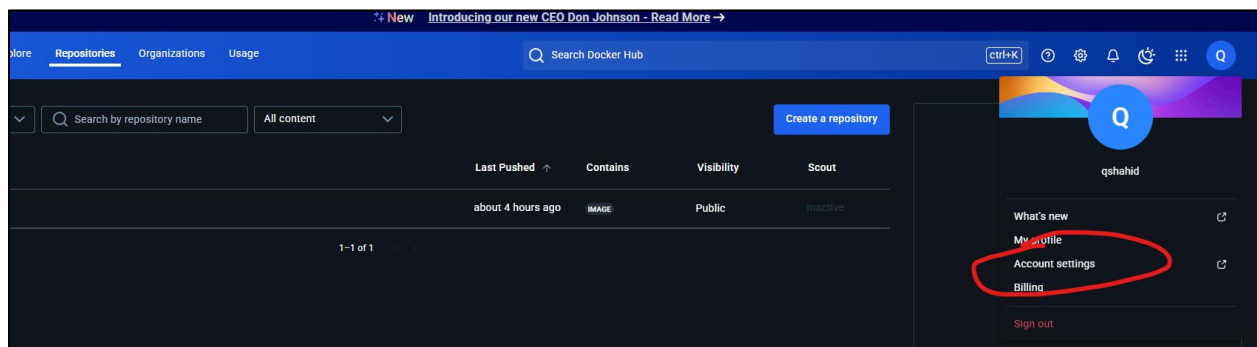
Description ?

Create

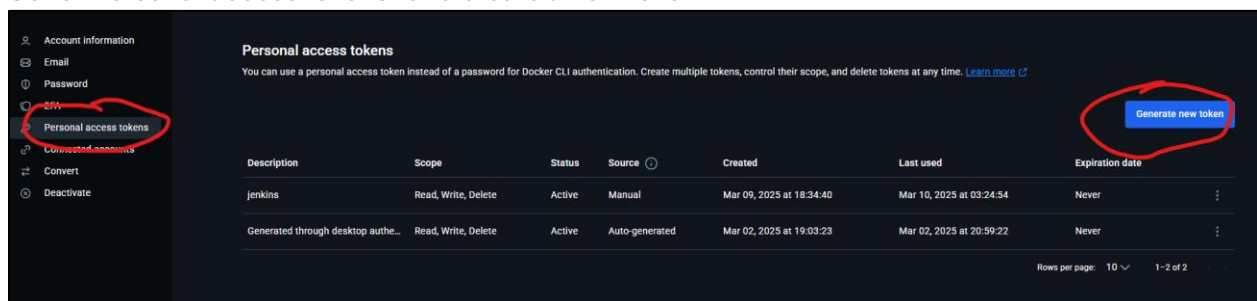
This will allow us to deploy to our cluster even though we are not on the same instance. Jenkins can use Kubernetes CLI with this file to connect to our cluster on the other EC2 instance.

Second, lets add our Docker Hub credentials:

First, you need to go to Docker Hub and login to your account. Go to “Account settings.”



Go to “Personal access tokens” and create a new token.



Copy these settings:

Access permissions – Read, Write, Delete

Personal access tokens / New access token

### Create access token

A personal access token is similar to a password except you can have many tokens and revoke access to each one at any time. [Learn more](#)

Access token description  
jenkins2

Expiration date  
None

Optional  
Access permissions  
Read, Write, Delete

Read, Write, Delete tokens allow you to manage your repositories.

Cancel Generate

Click generate. After that, it will give you a password. Copy this and save it somewhere as we will need it here soon. This will only show once, so be careful and make sure to save it somewhere.

Personal access tokens / New access token

### Copy access token

Use this token as a password when you sign in from the Docker CLI client. [Learn more](#)

Make sure you copy your personal access token now. Your personal access token is only displayed once. It isn't stored and can't be retrieved later.

Access token description  
jenkins2

Expires on  
Never

Access permissions  
Read, Write, Delete

To use the access token from your Docker CLI client:

- Run  

```
$ docker login -u qshahid
```

[Copy](#)
- At the password prompt, enter the personal access token  

```
docker login -u qshahid -p [token]
```

[Copy](#)

[Back to access tokens](#)

Now, go back to Jenkins and create another credential:

Make it as a “Username with password”. Then, add your Docker Hub username and take the password you just copied from the Personal Access Token and paste it into the password field. Then, give it this id:  
dockerhub\_credentials

Save this credential. Here is an example:

New credentials

Kind

Username with password

Scope ?

Global (Jenkins, nodes, items, all child items, etc)

Username ?

qshahid

☐ Treat username as secret ?

Password ?

\*\*\*\*\*

ID ?

dockerhub\_credentials

Description ?

Create

Now, we also need to add our “db\_secret.json” file to Jenkins. Follow the same steps you did for the kubeconfig file.

Select your custom “db\_secret.json” (the one where you have replaced the example details with the real details of your database login information) as the file and use the id “db\_secret\_file”.

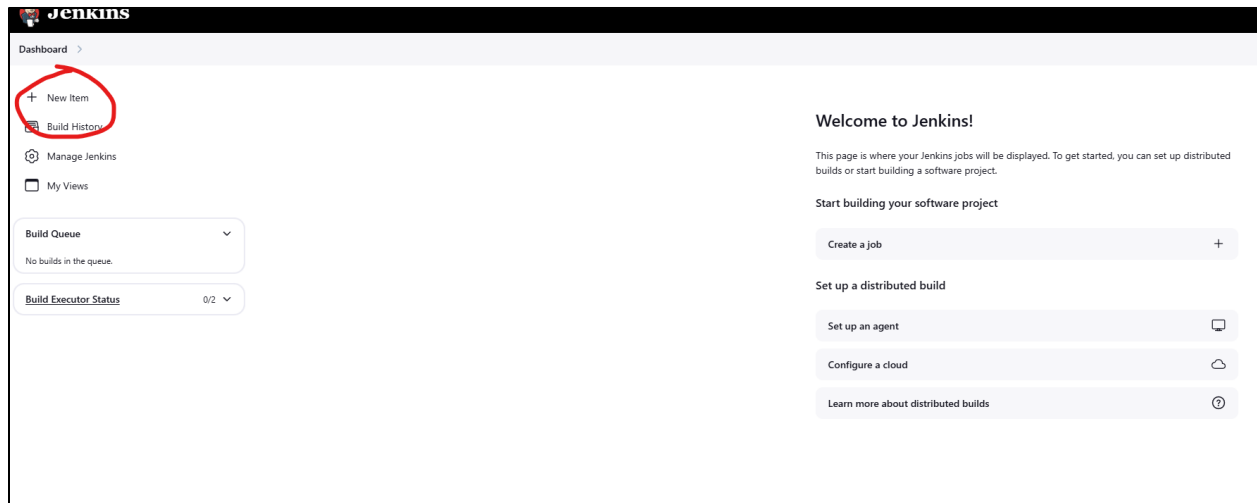
|                                                                                                    |                |             |                                                                                       |
|----------------------------------------------------------------------------------------------------|----------------|-------------|---------------------------------------------------------------------------------------|
|  db_secret_file | db_secret.json | Secret file |  |
|----------------------------------------------------------------------------------------------------|----------------|-------------|---------------------------------------------------------------------------------------|

Overall, you should have this setup in your credentials in Jenkins:

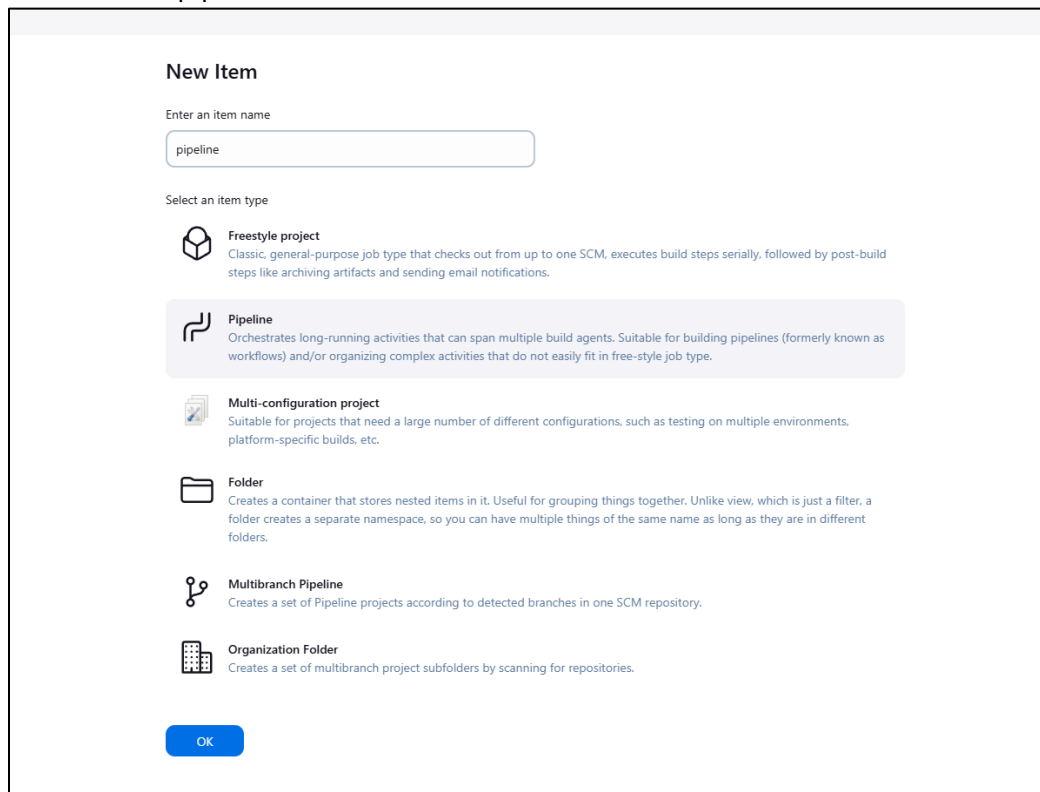
| Global credentials (unrestricted)                                                                          |                |                        |
|------------------------------------------------------------------------------------------------------------|----------------|------------------------|
| Credentials that should be available irrespective of domain specification to requirements matching.        |                |                        |
| ID                                                                                                         | Name           | Kind                   |
|  kubeconfig_credentials | cluster.yaml   | Secret file            |
|  dockerhub_credentials  | qshahid/*****  | Username with password |
|  db_secret_file         | db_secret.json | Secret file            |

Finally, we are ready to create our pipeline. Go back to the Jenkins dashboard and create a new item:





Create a new pipeline.



This will give you a bunch of options. We want to copy these settings:

The screenshot shows the Jenkins Pipeline Configuration page. The 'Triggers' section has 'GitHub hook trigger for GITScm polling' selected. The 'Pipeline' section has 'Definition' set to 'Pipeline script from SCM'. The 'SCM' section has 'Git' selected. The 'Repository URL' is set to 'https://github.com/qasimshahid/qshahid-swe645-asst2/'. The 'Branches to build' section has '\*/main' entered. The 'Repository browser' is set to '(Auto)'. The 'Save' and 'Apply' buttons are at the bottom.

Namely, do the following:

Select “GitHub hook trigger for GITScm polling” as a trigger

Pipeline definition should be from “Pipeline script from SCM”. This means it will use our Jenkinsfile in our GitHub repository. By default, the name of the file is Jenkinsfile, so we don’t need to change it.

Add the link to your GitHub repository.

Change the branch to “main”. This tells Jenkins to build the main branch. This is important because it is no longer named “master,” so make sure you enter “main” here.

After all of that, save the pipeline.

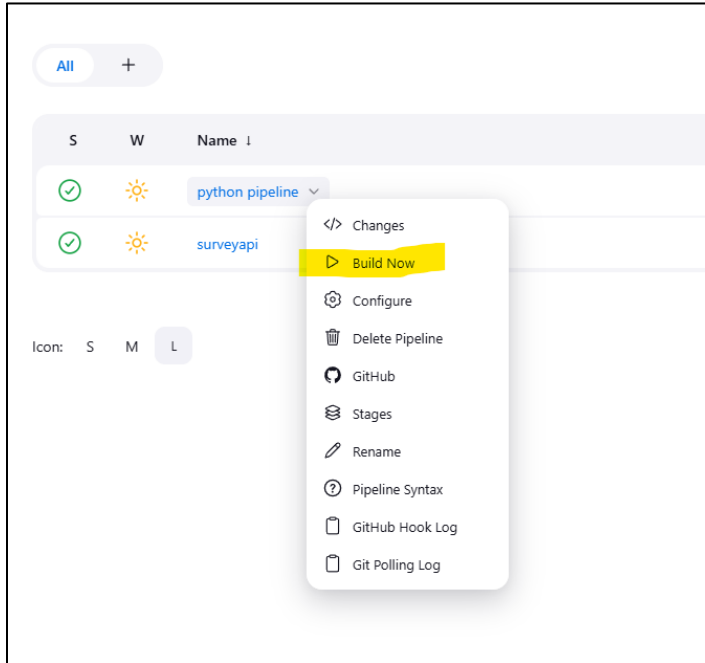
Congratulations, we are ready to do our first build. Everything should be setup properly now.

## Running the Application

Assuming you followed all the rest of the steps, you should be ready to build and deploy the app to your cluster.

You can do these two ways.

Manually via Jenkins:



Alternatively, if you were able to correctly setup your webhook from your GitHub repository, then you can simply push updates to your main branch and the pipeline will be triggered automatically.

IF YOU HAVE ANY ISSUES WITH THE DOCKER BUILD STEP:

Execute this command and then restart the Jenkins server.

```
sudo usermod -a -G docker Jenkins  
sudo reboot
```

Once you either push a change to GitHub or manually trigger the build through Jenkins, you should see this in the build output console:

```
[Pipeline] withKubeConfig
[Pipeline] {
[Pipeline] sh
+ kubectl apply -f ./deployment.yaml
deployment.apps/pythonsurvey-deployment unchanged
[Pipeline] sh
+ kubectl apply -f ./service.yaml
service/pythonsurvey-service unchanged
[Pipeline] sh
+ kubectl rollout restart deployment/pythonsurvey-deployment
deployment.apps/pythonsurvey-deployment restarted
[Pipeline] }
[kubernetes-cli] kubectl configuration cleaned up
[Pipeline] // withKubeConfig
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] cleanup
[MS-CLEANUP] Deleting project workspace...
[MS-CLEANUP] Deferred wipeout is used...
[MS-CLEANUP] done
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Here you can see our build succeeding (after a couple mistakes I made when recreating this, if you follow the guide, you should not have these.)

 **python pipeline**

Permalinks

- [Last build \(#20\), 10 hr ago](#)
- [Last stable build \(#20\), 10 hr ago](#)
- [Last successful build \(#20\), 10 hr ago](#)
- [Last failed build \(#4\), 11 hr ago](#)
- [Last unsuccessful build \(#4\), 11 hr ago](#)
- [Last completed build \(#20\), 10 hr ago](#)

New image successfully pushed to Docker Hub under my account.

**qshahid/pythonsurvey** 

Last pushed about 11 hours ago • Repository size: 645.3 MB

[Add a description](#)  

[Add a category](#)  

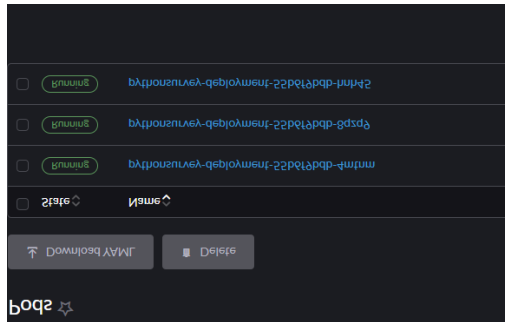
**General** Tags Image Management **BETA** Collaborators Webhooks Settings

**Tags**

This repository contains 18 tag(s).

| Tag                                                                                        | OS                                                                                  | Type  | Pulled          |
|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------|-----------------|
|  latest |  | Image | less than 1 day |
|  20     |  | Image | less than 1 day |

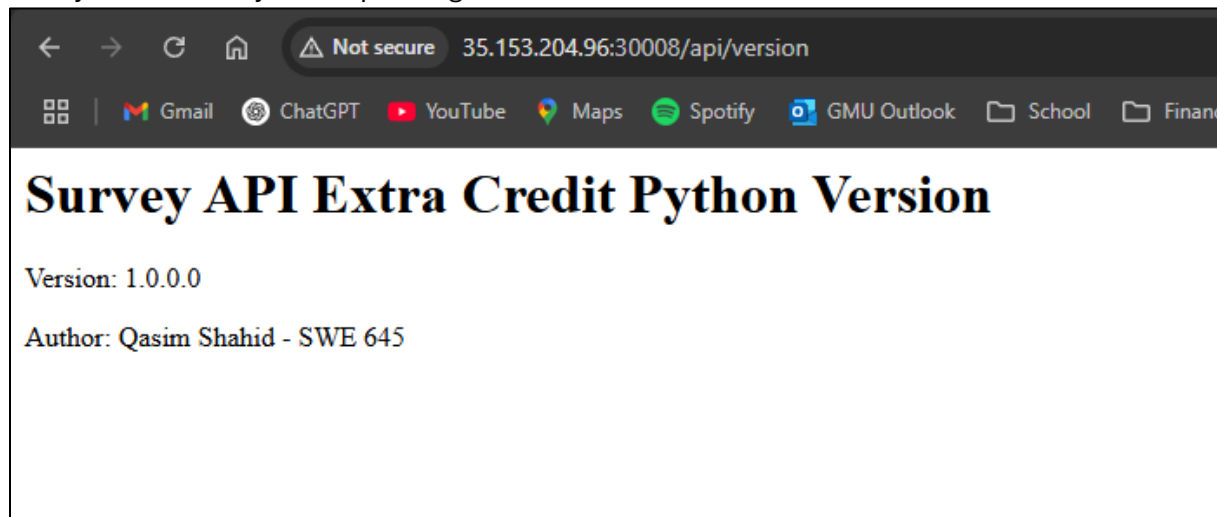
In Rancher, you can see the 3 pods / replicas we created as a part of our deployment.



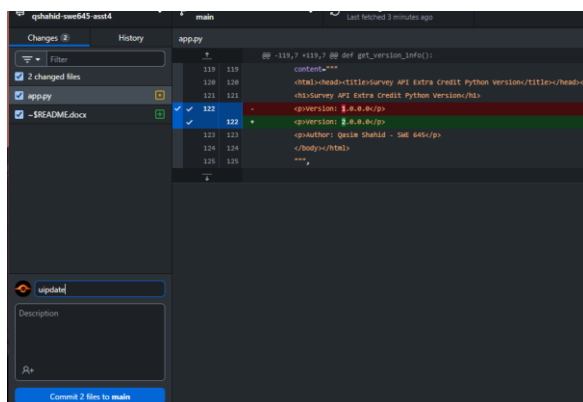
You can access these at the Rancher Instance's public elastic IP. The NodePort service I used hosts the API on the 30008 port. So, you can go to:

<http://{{rancher-ec2-ip}}:30008/api/version>

Here you can see my API responding via some HTML when we ask for the version.



I'll now push a small change to GitHub, let's see what happens. We will change version to 2.0.0.0.

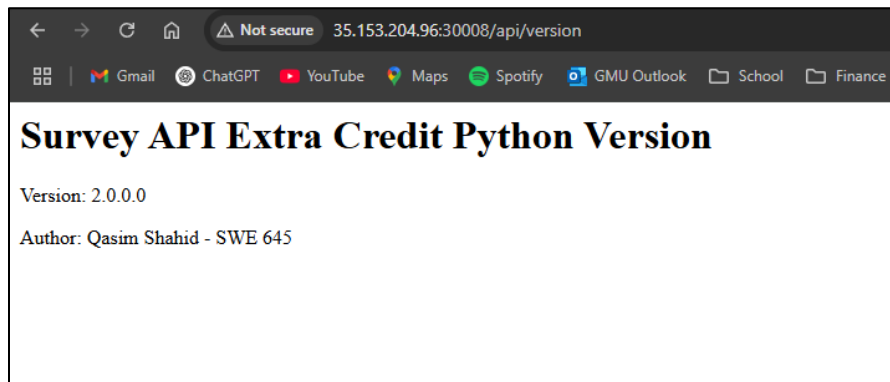


```

[Pipeline] {
[Pipeline] withKubeConfig
[Pipeline] {
[Pipeline] sh
+ kubectl apply -f ./deployment.yaml
deployment.apps/pythonsurvey-deployment unchanged
[Pipeline] sh
+ kubectl apply -f ./service.yaml
service/pythonsurvey-service unchanged
[Pipeline] sh
+ kubectl rollout restart deployment/pythonsurvey-deployment
deployment.apps/pythonsurvey-deployment restarted
[Pipeline] }
[kubernetes-cli] kubectl configuration cleaned up
[Pipeline] // withKubeConfig
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] cleanWs
[WS-CLEANUP] Deleting project workspace...
[WS-CLEANUP] Deferred wipeout is used...
[WS-CLEANUP] done
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```

Let's go to <http://35.153.204.96:30008/api/version> again and see if our pipeline deployed our change.



You can see that it worked. We are now on version 2.0.0.0, which we just pushed. Our pipeline listened to the push event and automatically deployed the change to our cluster.

NOTE: I won't be testing the API for doing CRUD operations, but this is covered in the demo\_video.mp4 file which will be included with the submission. In that, we go through all the CRUD operations on our APIs and make sure they are being persisted in our database.

## Python Code Design and Implementation

In this section, I'll go over some of my source code in the app.py file and explain how I created this API.

```
app.py
1  from fastapi import FastAPI, HTTPException
2  from fastapi.responses import HTMLResponse
3  from pydantic import BaseModel, EmailStr, Field, constr
4  from peewee import *
5  from datetime import date
6  import json
7  from typing import Optional
8
9  # Load secrets
10 with open('db_secret.json') as f:
11     secrets = json.load(f)
12
13 print(f"Connecting to database with host: {secrets['DB_HOST']} and port: {secrets['DB_PORT']}")
14
15 # MySQL DB
16 db = MySQLDatabase(
17     secrets['DB_NAME'],
18     user=secrets['DB_USER'],
19     password=secrets['DB_PASSWORD'],
20     host=secrets['DB_HOST'],
21     port=secrets['DB_PORT']
22 )
23
```

In this part of the code, I imported the key libraries needed to build the API. FastAPI is the main web framework used to define routes and handle requests. Pydantic is used to define request schemas and validate incoming JSON data cleanly (such as minimum length requirements for a string in JSON). Peewee is a lightweight ORM that handles communication with the MySQL database, similar to Hibernate in Java. I also used json to load database credentials from a db\_secret.json file, which helps keep sensitive info like passwords out of the main codebase.

```

print(f"Connecting to database with host: {secrets['DB_HOST']} and port: {secrets['DB_PORT']}")

# MySQL DB
db = MySQLDatabase(
    secrets['DB_NAME'],
    user=secrets['DB_USER'],
    password=secrets['DB_PASSWORD'],
    host=secrets['DB_HOST'],
    port=secrets['DB_PORT']
)

# Peewee ORM model
class Survey(Model):
    firstName = CharField()
    lastName = CharField()
    streetAddress = CharField()
    city = CharField()
    state = CharField()
    zipcode = CharField()
    telephone = CharField()
    email = CharField()
    dateOfSurvey = DateField()
    likedMost = CharField(null=True)
    interestSource = CharField(null=True)
    recommendLikelihood = CharField(null=True)
    additionalComments = TextField(null=True)

    class Meta:
        database = db

# Pydantic request model (v1)
class SurveyRequest(BaseModel):
    firstName: constr(min_length=1, max_length=255) = Field(...)
    lastName: constr(min_length=1, max_length=255) = Field(...)
    streetAddress: constr(min_length=1, max_length=255) = Field(...)
    city: constr(min_length=1, max_length=255) = Field(...)
    state: constr(min_length=1, max_length=255) = Field(...)
    zipcode: constr(min_length=1, max_length=20) = Field(...)
    telephone: constr(min_length=1, max_length=20) = Field(...)
    email: EmailStr = Field(...)
    dateOfSurvey: date = Field(...)
    likedMost: constr(max_length=255) = Field(None)
    interestSource: constr(max_length=255) = Field(None)
    recommendLikelihood: constr(max_length=255) = Field(None)
    additionalComments: constr(max_length=1500) = Field(None)

# FastAPI app
app = FastAPI()
db.connect()
db.create_tables([Survey])

```

This section sets up the database connection, defines the schema for the survey table, and gets everything ready for the FastAPI app to run. The `print()` is just a simple way to confirm the app is connecting to the right database host and port.

I used Peewee to connect to the MySQL database using credentials from the `db_secret.json` file (which is stored in Jenkins as a secret file and copied to the working directory). The `Survey` model defines the structure of the database table. Right after that, I created a Pydantic model called `SurveyRequest` to validate incoming data before saving it, so if anything is missing or invalid, FastAPI will reject it automatically.

Finally, I call `db.connect()` to open the DB connection and `db.create_tables([Survey])` to make sure the table gets created when the app starts. Then `app = FastAPI()` spins up the actual API server.



```

63
64 # Routes
65 @app.post("/api/surveys")
66 def create_survey(survey: SurveyRequest):
67     try:
68         validate_survey_input(survey)
69         new_survey = Survey.create(**survey.dict())
70         return {"id": new_survey.id, "message": "Survey created successfully"}
71     except Exception as e:
72         raise HTTPException(status_code=500, detail=str(e))
73
74 @app.get("/api/surveys")
75 def get_all_surveys():
76     try:
77         return [s.__data__ for s in Survey.select()]
78     except Exception as e:
79         raise HTTPException(status_code=500, detail=str(e))
80
81 @app.get("/api/surveys/{id}")
82 def get_survey_by_id(id: int):
83     try:
84         survey = Survey.get(Survey.id == id)
85         return survey.__data__
86     except Survey.DoesNotExist:
87         raise HTTPException(status_code=404, detail="Survey not found")
88     except Exception as e:
89         raise HTTPException(status_code=500, detail=str(e))
90
91 @app.put("/api/surveys/{id}")
92 def update_survey(id: int, survey: SurveyRequest):
93     try:
94         validate_survey_input(survey)
95         existing = Survey.get(Survey.id == id)
96         for k, v in survey.dict().items():
97             setattr(existing, k, v)
98         existing.save()
99         return {"message": "Survey updated successfully"}
100     except Survey.DoesNotExist:
101         raise HTTPException(status_code=404, detail="Survey not found")
102     except Exception as e:
103         raise HTTPException(status_code=500, detail=str(e))
104
105 @app.delete("/api/surveys/{id}")
106 def delete_survey(id: int):
107     try:
108         survey = Survey.get(Survey.id == id)
109         survey.delete_instance()
110         return {"message": "Survey deleted successfully"}
111     except Survey.DoesNotExist:
112         raise HTTPException(status_code=404, detail="Survey not found")
113     except Exception as e:
114         raise HTTPException(status_code=500, detail=str(e))
115
116 @app.get("/api/version")
117 def get_version_info():
118     return HTMLResponse(
119         content="""
120         <html><head><title>Survey API Extra Credit Python Version</title></head><body>
121         <h1>Survey API Extra Credit Python Version</h1>
122         <p>Version: 2.0.0.0</p>
123         <p>Author: Qasim Shahid - SWE 645</p>
124         </body></html>
125         """,
126         status_code=200
127     )
128
129 # Manual validations
130 def validate_survey_input(survey: SurveyRequest):
131     VALID_LIKED = ["students", "location", "campus", "atmosphere", "dorm rooms", "sports"]
132     VALID_INTEREST = ["friends", "television", "internet", "other"]
133     VALID_RECOMMEND = ["very likely", "likely", "unlikely"]
134
135     if survey.likedMost and survey.likedMost.lower().strip() not in VALID_LIKED:
136         raise HTTPException(400, f"Invalid liked_most. Must be one of {VALID_LIKED}")
137
138     if survey.interestSource and survey.interestSource.lower().strip() not in VALID_INTEREST:
139         raise HTTPException(400, f"Invalid interest_source. Must be one of {VALID_INTEREST}")
140
141     if survey.recommendLikelihood and survey.recommendLikelihood.lower().strip() not in VALID_RECOMMEND:
142         raise HTTPException(400, f"Invalid recommend_likelihood. Must be one of {VALID_RECOMMEND}")
143

```

This section defines all the main API routes using FastAPI. Each route corresponds to a standard CRUD operation for the survey records stored in the database.

The `create_survey` function handles POST requests and is used to insert a new survey into the database. It takes a `SurveyRequest` object, which is automatically validated by Pydantic. Inside the function, I call a helper called `validate_survey_input()` to run extra validation on specific fields like `likedMost`, `interestSource`, and `recommendLikelihood`. Then, I use `Survey.create(**survey.dict())` to insert the data into the database. This line takes the dictionary version of the validated survey object and unpacks it into keyword arguments for Peewee's `create()` method, which is a cleaner way to insert all fields at once. The `**` is Python's unpacking operator, it takes a dictionary and spreads its key-value pairs into arguments. So if `survey.dict()` returns `{"firstName": "Qasim", "email": "qasim@example.com"}`, then `**survey.dict()` turns into `firstName="Qasim", email="qasim@example.com"` in the function call. If any error happens in the process, it's caught by the `try/except`, and FastAPI returns a proper 500 error with a description of what went wrong.

The `get_all_surveys` route is a GET request that fetches all the surveys from the database using `Survey.select()`. Each result is returned as a dictionary via `s.__data__`, which exposes the Peewee model data in a JSON-friendly format. If there's a problem with the database query, it returns a 500 error.

For fetching a single survey, the `get_survey_by_id` route uses `Survey.get()` with a filter on the ID. If the record doesn't exist, Peewee throws a `DoesNotExist` exception, and I handle that specifically by returning a 404 response with a "Survey not found" message.

The `update_survey` route handles PUT requests for updating an existing record. It also uses `validate_survey_input()` to re-check the fields before updating. Then it gets the existing database row, loops through the keys from the request object, and updates each field on the Peewee model using `setattr()`. After setting all the values, it saves the record with `existing.save()`. Again, it handles missing records and general errors gracefully using `try/except`.

The `delete_survey` route deletes a record if it exists, using `delete_instance()`, and returns a success message — or a 404 if the record wasn't found.

Lastly, there's a simple `/api/version` route that returns an HTML response with version info for the API, just to show it's running and display some author details.

And at the bottom, the `validate_survey_input()` function checks that specific fields contain valid values by comparing them to hardcoded lists. If they don't match, it raises an HTTP 400 error so the user gets a clear message explaining what was wrong. This gives me more control over the data we add to our database.

Overall, this is the full application. It includes all the same features as my API from Assignment 3, which I built using Java and Spring Web. But since this version is in Python, I was able to take advantage of packages like FastAPI, Pydantic, and Peewee to build everything in a single file, with way less code and complexity.

#### References:

<https://docs.docker.com/security/for-developers/access-tokens/>  
<https://kubernetes.io/>  
<https://plugins.jenkins.io/pipeline-github/>  
<https://www.cprime.com/resources/blog/how-to-integrate-jenkins-github/>  
<https://www.rancher.com/>  
<https://ranchermanager.docs.rancher.com/>  
<https://plugins.jenkins.io/kubernetes-cli/>  
<https://www.tutorialspoint.com/peewee/index.htm>  
<https://docs.pydantic.dev/latest/concepts/models/>  
<https://www.geeksforgeeks.org/creating-first-rest-api-with-fastapi/>  
<https://realpython.com/fastapi-python-web-apis/>  
<https://realpython.com/python-pydantic>

#### Lessons Learned:

This assignment, I learned how to deploy REST APIs to a Kubernetes cluster using Python and FastAPI. In the previous assignment, we also deployed a REST API, but we used Maven and Java, with Spring Web. This time, I used some of the more popular Python packages, such as Peewee for ORM (similar to Java's Hibernate), FastAPI for building the API itself, and Pydantic for request validation.

The overall process was very similar: containerize the app with Docker, set up a Kubernetes deployment and service, and expose the API - but working with FastAPI required a lot less boilerplate than Java and Maven. The syntax was cleaner, and setting up models, routes, and validation felt more lightweight and easier to follow. I was able to put all my API code in just one Python file!